AEGIS

Aeternix Engineering & Governance Intelligence System

User Manual & Reference Guide

Version 5.0.0

Aeternix (aeternix.tech)
March 2026

Table of Contents

Chapter 1: Introduction

> *Standards defined in Markdown, enforced by AI, validated autonomously.*

AEGIS (Aeternix Engineering & Governance Intelligence System) is a comprehensive AI-powered development framework that transforms a solo developer or small team into a full-stack engineering organization.

Built on the SDLC+AI methodology where Markdown is the Source of Truth and AI is the Engine, AEGIS provides 16 specialized skills, 8 AI personas, 3 adaptive tracks, party mode collaboration, a skill marketplace, and a complete marketing content pipeline.

1.1 What is AEGIS?

AEGIS is a collection of Claude Skills that work together as an intelligent development team. Each skill is a structured playbook that tells Claude exactly how to perform a specific task, from writing specs to reviewing code to scanning for security vulnerabilities.

Unlike general-purpose AI assistants, AEGIS uses context isolation through personas. When you invoke Vigil (the Code Guardian), Claude focuses exclusively on code quality. When you invoke Havoc (the Red Team), Claude becomes adversarial and challenges every decision. This prevents the context pollution that degrades AI output quality.

1.2 Core Philosophy

- Markdown is the Source of Truth — specs, standards, and plans live in Markdown files that both humans and AI can read
- AI is the Engine — Claude executes against these specs with specialized skills
- Infinite Loop — PLAN → EXECUTE → VERIFY → FEEDBACK → loop back to PLAN
- Scale-Adaptive — automatically adjusts process depth to project complexity
- Context Isolation — each persona owns specific skills, preventing cross-contamination

1.3 AEGIS vs BMAD

AEGIS was designed with awareness of the BMAD Method (35k+ GitHub stars), adopting its best ideas while surpassing it in quality gate depth, marketing automation, and ecosystem tooling.

Feature	AEGIS	BMAD
Skills/Capabilities	16 skills	12 agents
Quality Gate Depth	7 VERIFY/FEEDBACK skills	2 (review + QA)
Marketing Pipeline	5 skills, end-to-end	None
Test Architecture	Risk-based + chaos + gates	TEA module (8 workflows)
Skill Marketplace	Registry + ratings + collections	None
Bilingual	EN + TH native	EN + CN (docs only)
Lifecycle	Infinite loop	Linear 4-phase

Chapter 2: Installation

2.1 Prerequisites

- Claude Desktop App, Claude.ai, Claude Code, or Cowork with Skills support
- Or any Claude-compatible environment that supports .skill files

2.2 Method 1: Drag & Drop (Recommended)

1. Download AEGIS-v5.0.0.zip
2. Unzip to get 16 .skill files + README + LICENSE
3. Open Claude → Settings → Skills
4. Drag each .skill file into the Skills area
5. Skills activate automatically based on trigger phrases

2.3 Method 2: Manual Install (Claude Code / Cowork)

Copy skill folders into your skills directory:

```
SKILLS_DIR="/path/to/your/skills/user"
unzip AEGIS-v5.0.0.zip -d $SKILLS_DIR/../
```

Verify installation:

```
ls $SKILLS_DIR/*/SKILL.md | wc -l  # Should show 16
```

2.4 First Run

After installation, invoke the navigator to verify everything works:

```
"Navi, what should I do next?"
```

Navi will scan your project, detect its state, and recommend the first action.

Chapter 3: Your AI Team — Personas

AEGIS provides 8 specialized AI personas. Each has a distinct personality, communication style, and set of owned skills. When you invoke a persona, Claude adopts that character and focuses exclusively on their domain.

Icon	Name	Role	Owns	Phase
🧭	Navi	Navigator / Project Guide	project-navigator	META
📐	Sage	Spec Architect / Planner	spec-kit, code-standards	PLAN
🖌️	Pixel	UX Designer	wireframes, user flows, UX specs	PLAN
⚡	Bolt	Developer / Builder	autonomous-coding, api-docs	EXECUTE
🛡️	Vigil	Code Guardian / Tester	code-review, code-coverage	VERIFY
🔴	Havoc	Red Team / Security	adversarial-review, security-audit	VERIFY
🔧	Forge	DevOps / Maintainer	git-workflow, tech-debt, sprint, retro, course	FEEDBACK
🎨	Muse	Creative Strategist	trend, content, imagegen, blast, gdrive	MARKETING

3.1 How to Invoke

Single Persona

Just say the name with your request:

```
"Sage, create a spec for user authentication"
"เรียก Vigil — review this PR"
"Havoc, break my architecture"
```

Switch Persona

```
"Switch to Bolt"  /  "สลับเป็น Bolt"
"Bring in Vigil"  /  "เรียก Vigil มา"
```

Dismiss Persona

```
"Drop persona"  /  "กลับปกติ"
```

3.2 Persona Details

Navi — The Navigator

Calm, strategic, big-picture thinker. Navi never writes code or reviews it. Navi's only job is to tell you WHERE you are and WHAT to do next.

- Starts with project state assessment
- Gives numbered action items with specific skill/persona recommendations
- References specific personas by name for handoff
- Catchphrase: "Let me check where we are..."

Sage — The Spec Architect

Methodical, precise, slightly perfectionist. Asks a LOT of questions before writing anything.

- Structures output with clear hierarchies (L1 → L2 → L3 → L4 Markdown)
- Uses requirement IDs (FR-001, NFR-003) religiously
- Pushes back on vague requirements: "What does 'fast' mean? Give me a number."
- Catchphrase: "Before we write a single line..."

Pixel — The UX Designer

Empathetic, user-obsessed, visual thinker. Always asks "but what does the USER experience?"

- Produces wireframe specs, user journey maps, accessibility checklists
- Challenges UI complexity: "Do we really need this modal?"
- Catchphrase: "From the user's perspective..."

Bolt — The Developer

Fast, pragmatic, production-focused. Bolt wants to ship.

- Follows specs precisely but suggests improvements when seen
- Shows code first, explains after
- Catchphrase: "Building it now..."

Vigil — The Code Guardian

Thorough, skeptical but constructive, detail-oriented.

- 5-pass structured review: Correctness → Security → Performance → Maintainability → SDD
- Celebrates good code as much as flagging bad code
- Catchphrase: "I found 3 issues and 2 things I really like..."

Havoc — The Red Team

Provocative, relentless, zero-tolerance for hand-waving. Assumes EVERYTHING is broken until proven otherwise.

- Uses BREAK framework: Better Alternative? Reversibility? Edge Cases? Assumptions? Kill Scenario?
- Generates stress test and kill scenarios

- Catchphrase: "Let me break this..."

Forge — The DevOps Engineer

Systematic, efficiency-obsessed, automate-everything mindset.

- Owns the most skills (5): git-workflow, tech-debt, sprint, retro, course-correction
- Thinks in terms of sprint 20, not just sprint 1
- Catchphrase: "Let’s automate this..."

Muse — The Creative Strategist

Creative, trend-savvy, data-informed. Thinks in campaigns, not posts.

- Orchestrates full marketing pipeline: research → content → visuals → delivery
- Catchphrase: "Here’s what’s trending..."

Chapter 4: Skills Reference

AEGIS contains 16 skills organized by SDLC phase. Each skill is a structured playbook that Claude reads and follows.

#	Skill	Purpose	Phase
0	project-navigator	Meta-orchestrator — scan, recommend, guide	META
1	code-standards	Coding standards definition & enforcement	PLAN
2	code-review	5-pass structured code review	VERIFY
3	adversarial-review	Devil's advocate with BREAK framework	VERIFY
4	security-audit	OWASP Top 10, deps, secrets, infra	VERIFY
5	code-coverage	Coverage analysis + test generation	VERIFY
6	git-workflow	Branching, commits, PRs, changelogs	FEEDBACK
7	api-docs	OpenAPI generation + drift detection	EXECUTE
8	tech-debt-tracker	TODO scan, complexity, debt score	FEEDBACK
9	sprint-tracker	Sprint planning, stories, velocity	FEEDBACK
10	retrospective	Sprint/epic retro with action items	FEEDBACK
11	course-correction	Mid-sprint scope change management	FEEDBACK
12	test-architect	Enterprise test strategy & quality gates	VERIFY
13	aegis-builder	Create custom skills, personas, modules	META
14	skill-marketplace	Discover, share, install skills	META
15	ai-personas	8 AI personas + party mode	META

Chapter 5: Scale-Adaptive Tracks

AEGIS automatically adjusts the development process based on project complexity. Navi detects the appropriate track during project scanning.

5.1 Quick Track

Stories: 1–15 | Team: Solo developer | Best for: bug fixes, small features

Minimal process. Spec → Build → Review → Done.

Step	Action
1	Sage: Quick spec (or skip if scope is obvious)
2	Bolt: Implement
3	Vigil: Code review
4	Done — merge and deploy

5.2 Standard Track

Stories: 10–50 | Team: 2–5 people | Best for: products, APIs, platforms

Full PLAN → VERIFY cycle, minus adversarial review.

Phase	Actions
PLAN	Sage: full spec + code-standards | Pixel: UX design (if UI)
EXECUTE	Bolt: implement per story | api-docs: generate
VERIFY	Vigil: code-review + coverage | Havoc: security-audit
FEEDBACK	Forge: git-workflow + tech-debt + sprint + retro

5.3 Enterprise Track

Stories: 30+ | Team: 5+ people | Best for: multi-team, compliance, high-stakes

All 16 skills, no shortcuts. Includes adversarial review and test architecture.

Phase	Actions
PLAN	Sage: full SDD spec + standards | Pixel: UX + accessibility
EXECUTE	Bolt: implement | api-docs | sprint-tracker: manage stories
VERIFY	Vigil: review + coverage | Havoc: adversarial + security | test-architect: quality gates
FEEDBACK	Forge: git + debt + sprint + retro + course-correction

Chapter 6: Common Workflows

6.1 Start a New Project

6. Install AEGIS skills
7. Say: "Navi, I'm starting a new project for [description]"
8. Navi scans, detects Empty state, recommends track
9. Say: "Sage, create a spec for [project]"
10. Sage asks questions, produces SPEC.md
11. Say: "Sage, set up coding standards"
12. Say: "Bolt, implement the first feature from spec"
13. Say: "Vigil, review Bolt's code"
14. Say: "Navi, what's next?" — Navi guides the loop

6.2 Review a Pull Request

15. Say: "Vigil, review this PR" (paste diff or file)
16. Vigil runs 5-pass review, produces severity-ranked findings
17. If critical security issues found: "Havoc, run security audit on auth module"
18. Fix findings, re-submit

6.3 Pre-Launch Readiness Check

19. Say: "Navi, is this project ready for production?"
20. Navi runs 9-criteria checklist across all skills
21. For Enterprise: "Havoc, adversarial review the architecture"
22. "Forge, what's our tech debt score?"
23. Fix blockers, re-check

6.4 Sprint Planning

24. Say: "Forge, initialize sprint planning from our spec"
25. Forge generates sprint-status.yaml with epics, stories, and estimates
26. "Forge, show sprint status" — see progress report
27. At sprint end: "Forge, run retrospective"

6.5 Handle Scope Change

28. Say: "Forge, the client wants to add [feature] mid-sprint"
29. Forge runs impact assessment (scope, dependencies, risk)
30. Presents options: Accept + defer, Extend sprint, Reject, or Minimal version
31. Decision logged for retrospective analysis

6.6 Run Marketing Pipeline

32. Say: "Muse, research trends in [industry]"
33. Muse produces trend report with content opportunities
34. "Muse, create a content calendar for next week"
35. "Muse, generate images for the posts"
36. "Muse, upload everything to Google Drive"

Or run the full pipeline in one shot:

```
"Muse, run the marketing blast for [topic]"
```

Chapter 7: Party Mode

> *Better decisions through diverse perspectives.*

Party mode brings multiple personas into one conversation for collaborative discussion. Each persona speaks in character, can agree or disagree with others, and action items are assigned to specific personas.

7.1 Invoke Party Mode

```
"Party mode: Sage, Bolt, and Havoc — discuss the auth architecture"
"ปาร์ตี้โหมด: ทุกคนมาประชุม — review launch readiness"
```

7.2 Party Mode Rules

37. Navi always speaks first (if included) — sets the context
38. Each persona only speaks within their domain
39. Disagreements are explicit: "I disagree with Bolt because..."
40. Action items are assigned: "Bolt: update api-docs. Vigil: add auth tests."
41. Navi always speaks last (if included) — summarizes decisions

7.3 Best Scenarios for Party Mode

Scenario	Personas	Why
Pre-launch readiness	Navi + Vigil + Havoc + Forge	Coverage, security, debt check
Architecture decision	Sage + Bolt + Havoc	Plan vs build vs break
Sprint retrospective	Navi + Forge + Vigil	Status, debt, quality trends
New feature planning	Sage + Pixel + Bolt	Spec + UX + feasibility
Post-incident review	Havoc + Vigil + Forge	What broke, untested, prevent

Chapter 8: Extending AEGIS

8.1 Create a Custom Skill

Use the aegis-builder skill to create new capabilities:

```
"aegis-builder, create a skill for database schema design"
```

Every skill needs: YAML frontmatter (name + description), EN + TH trigger phrases, output format template, persona assignment, and integration points with other skills.

8.2 Create a Custom Persona

Add domain-specific personas to your team:

```
"aegis-builder, add a persona called Shield for compliance"
```

Each persona needs: distinct voice, clear domain boundaries, owned skills, handoff protocol, and bilingual invocation.

8.3 Create a Domain Module

Bundle skills + personas for a specific industry:

- aegis-fintech — PCI compliance, fraud detection, regulatory reports
- aegis-healthtech — HIPAA audit, PHI scanner, clinical workflows
- aegis-gamedev — Game design doc, play-test, balance review

8.4 Skill Marketplace

Discover and install community-built skills:

```
"marketplace, find security skills"
"marketplace, install pci-compliance"
"marketplace, publish my-custom-skill"
```

Chapter 9: Quick Command Reference

Common commands organized by task. Both English and Thai triggers work.

9.1 Navigation & Status

Command	What Happens
"Navi" / "ทำอะไรต่อดี"	Scan project, recommend next action
"Navi, is this ready for production?"	Run 9-criteria readiness check
"Navi, what skills do I have?"	List all installed skills
"Drop persona" / "กลับปกติ"	Return to general Claude

9.2 Planning

Command	What Happens
"Sage, create a spec for [X]"	Full SDD spec with requirements
"Sage, set up coding standards"	Generate ESLint/Prettier/Ruff configs
"Pixel, design the user flow for [X]"	Wireframe spec + journey map

9.3 Building

Command	What Happens
"Bolt, implement [feature] from spec"	Build with autonomous-coding
"Bolt, update API docs"	Generate/update OpenAPI spec

9.4 Reviewing & Testing

Command	What Happens
"Vigil, review this code"	5-pass structured review
"Vigil, check test coverage"	Coverage analysis + suggest tests
"Havoc, break this" / "ท้าทาย"	Adversarial review with BREAK framework
"Havoc, security audit"	OWASP + deps + secrets + infra scan

9.5 DevOps & Process

Command	What Happens

"Forge, set up git workflow"	Branch strategy + PR template + hooks
"Forge, scan tech debt"	TODO + complexity + deps + debt score
"Forge, plan the sprint" / "แพลนงาน"	Generate sprint-status.yaml
"Forge, run retrospective" / "retro"	Sprint metrics + action items
"Forge, scope changed" / "เปลี่ยนสโคป"	Impact assessment + decision gate

9.6 Marketing

Command	What Happens
"Muse, research trends in [X]" / "เทรนด์"	Trend research report
"Muse, create content" / "สร้างคอนเทนต์"	Calendar + multi-platform copy
"Muse, marketing blast" / "รันไปป์ไลน์"	Full pipeline: research → content → images → upload

9.7 Party Mode

Command	What Happens
"Party mode: [names] — [topic]"	Multi-persona collaborative discussion
"ปาร์ตี้โหมด: ทุกคน — [topic]"	Same in Thai

Chapter 10: The SDLC+AI Infinite Loop

AEGIS operates as a continuous improvement cycle. Every phase feeds the next, and the loop never ends.

10.1 The Four Phases

Phase	What Happens	Personas	Skills
PLAN	Define what to build and how	Sage, Pixel	spec-kit, code-standards
EXECUTE	Build it, document it, track it	Bolt, Forge	autonomous-coding, api-docs, sprint-tracker
VERIFY	Review, test, audit, challenge	Vigil, Havoc	code-review, coverage, security, adversarial, test-architect
FEEDBACK	Measure, learn, improve	Forge, Navi	tech-debt, retro, course-correction, navigator

10.2 The Loop in Practice

42. Navi scans project state and recommends starting point
43. Sage creates spec with requirements and standards
44. Pixel designs user experience (if UI project)
45. Bolt implements features from spec
46. Vigil reviews code quality and test coverage
47. Havoc challenges design decisions (Enterprise track)
48. Forge tracks sprint progress, manages git workflow
49. Forge runs retrospective, identifies improvements
50. Navi re-scans — improvements feed into next PLAN cycle
51. Repeat until production-ready

10.3 The Handoff Chain

```
Navi → Sage → Pixel → Bolt → Vigil → Havoc → Forge → Navi (loop)
```

Each persona knows who comes next and explicitly recommends the handoff. You're never lost — just ask Navi.

Appendix A: Trigger Phrase Reference

Every persona and skill responds to both English and Thai triggers. Here are the most common ones:

Persona/Skill	English Triggers	Thai Triggers
Navi	"what’s next", "status", "navigator"	"ทำอะไรต่อ", "สถานะ"
Sage	"spec", "plan", "standards"	"วางแผน", "มาตรฐาน"
Pixel	"UX", "wireframe", "design"	"ออกแบบ UI"
Bolt	"build", "implement", "code"	"เขียนโค้ด"
Vigil	"review", "test", "coverage"	"รีวิว", "เทสต์"
Havoc	"break", "challenge", "red team"	"ท้าทาย", "หาจุดอ่อน"
Forge	"git", "debt", "sprint", "retro"	"จัดการ git", "หนี้"
Muse	"trends", "content", "marketing"	"เทรนด์", "คอนเทนต์"
Party mode	"party mode: [names]"	"ปาร์ตี้โหมด"